Dilution Breaks Extraction Before Detection: Evaluating LLM Steganalysis on Sparse Keyword Signals

Ari Holtzman and NeuriCo

Abstract

Sparse hidden signals are a realistic failure mode for text monitoring, but most prior evaluations of linguistic steganography focus on dense generation schemes, local edit rules, or binary cover-versus-stego classification. We study a narrower and practically important question: can current large language models detect and recover messages when the signal is diluted across long passages rather than concentrated in obvious local patterns? We build a controlled benchmark from WIKITEXT-2, IMDB, and AG NEWS with three encoding schemes: acrostics, dense keyword-trigger rules, and diluted keyword-trigger rules with distractor variants. We then evaluate GPT-4.1 and GPT-5 with fixed detection and extraction prompts on 72 synthetic examples, plus a small external sanity check.

The main result is consistent across both models. Exact recovery drops sharply under dilution, while detection degrades more slowly. For GPT-4.1, exact recovery falls from 0.75 on dense keyword encoding and 0.67 on acrostics to 0.25 on diluted keyword encoding. For GPT-5, the same ordering appears at 0.50, 0.58, and 0.17. The strongest confirmatory comparison is for GPT-4.1, where dense extraction significantly outperforms diluted extraction ($p = 0.039$ by Fisher's exact test).

These results show that current LLM-based monitors can often notice suspicious structure without reliably recovering the payload once the signal is sparse, dispersed, and mixed with ordinary text. That detection-extraction gap matters for misuse analysis and for benchmarking model-based steganalysis systems.

1 Introduction

Hidden communication in text becomes more concerning as text generation becomes cheaper, longer, and easier to automate. The hardest regime is not the obvious acrostic or the dense trigger rule. It is the low-density regime in which a message is spread across a long passage, mixed with distractors, and embedded in otherwise ordinary language. That is the regime a monitor would need to handle in practice.

Why does this problem matter? Recent work has made linguistic steganography more fluent and less statistically conspicuous by using masked language models, adaptive generative coding, and LLM-based generation Ueoka et al. [2021], Zhang et al. [2021], Lin et al. [2024], Wu et al. [2024]. At the same time, newer steganalysis work shows that language-model-based detectors can outperform older classifiers in some settings Yi et al. [2021], Bai et al. [2024]. But these two threads rarely meet on the specific question we care about here: whether present-day LLMs can recover a payload when the evidence for that payload is sparse and dispersed rather than locally obvious.

What is missing? Prior work mostly studies dense generation schemes, edit-based local changes, or cover-versus-stego detection Ueoka et al. [2021], Lin et al. [2024], Bai et al. [2024]. Frontier-model safety papers ask whether models can hide or pass messages under stronger coordination

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

assumptions, but they do not isolate dilution as an experimental variable in a corpus-grounded extraction benchmark Perry et al. [2025], Karpov et al. [2025], Zolkowski et al. [2025]. That leaves a direct benchmark gap: no existing evaluation in our review separately measures detection and exact payload recovery while sweeping from obvious structure to sparse keyword-conditioned rules.

Our approach. We address that gap with DILUTEDSTEGBENCH, a controlled benchmark built from WIKITEXT-2, IMDB, and AG NEWS. Each cover passage is turned into one of three schemes: ACROSTIC, KEYWORD-DENSE, or KEYWORD-DILUTED. The diluted setting keeps the same basic decoding rule as the dense setting but spreads trigger positions across a much longer passage and introduces distractor variants such as `signals` and `signal-based`. We evaluate GPT-4.1 and GPT-5 with separate prompts for detection and extraction, which lets us ask whether a model can suspect hidden structure before it can decode the six-letter secret.

What do we find? The main pattern is stable across both models. GPT-4.1 reaches exact recovery rates of 0.67 on ACROSTIC, 0.75 on KEYWORD-DENSE, and 0.25 on KEYWORD-DILUTED. GPT-5 shows the same ordering at 0.58, 0.50, and 0.17. Detection remains materially stronger than exact recovery in the diluted setting: for example, GPT-4.1 still achieves 0.79 detection accuracy on KEYWORD-DILUTED while recovering only 25% of secrets exactly.

Our contributions are:

- We introduce a corpus-grounded benchmark that isolates signal dilution while separating detection from extraction.
- We evaluate two current OpenAI API models under fixed prompts and show that diluted keyword signals are consistently harder to recover than obvious or dense schemes.
- We quantify corpus effects, statistical uncertainty, and pairwise differences, including a significant dense-versus-diluted gap for GPT-4.1.
- We document practical failure modes showing that current LLM monitors often detect suspicious structure without recovering the payload itself.

The rest of the paper reviews related work in section 2, describes the benchmark and evaluation protocol in section 3, presents quantitative results in section 4, and discusses implications and limitations in section 5.

2 Related Work

Generative and edit-based linguistic steganography. Earlier text steganography work focused on dense encoding schemes that steer generation or edit local tokens while preserving fluency. Adaptive dynamic grouping gives a formal generative baseline for secure text steganography Zhang et al. [2021]. Edit-based methods using masked language models show that sparse local substitutions can be much harder to detect than naïve baselines Ueoka et al. [2021]. More recent LLM-era systems improve quality and capacity with in-context generation or black-box prompting, including zero-shot generative steganography and keyword-set methods for large models Lin et al. [2024], Wu et al. [2024]. Raj-Sankar and Rajagopalan argue that density, detectability, and decodability should be treated as linked design variables Raj-Sankar and Rajagopalan [2023]. Our work builds on that framing, but instead of proposing a new hiding method, we isolate dilution as an evaluation axis for present-day model-based steganalysis.

Steganalysis with language models. A second thread studies whether learned language representations help detect hidden text. Yi et al. show that pretrained language-model features improve steganalysis over older RNN and CNN baselines in several settings Yi et al. [2021]. Bai et al. extend this idea by using LLMs directly as steganalyzers and report large gains over classical detectors on dense or structurally simpler tasks Bai et al. [2024]. These results motivate our use of prompted LLMs as black-box analyzers, but they do not answer whether the same models can recover a sparse secret when evidence is scattered across long context. We focus on that harder extraction setting while still measuring binary detection.

Robustness and frontier-model covert capability. Recent work has shifted from classical steganography metrics toward robustness and safety questions. Perry et al. distinguish local and semantic robustness and show that some LLM-based schemes survive paraphrase or light edits better than watermark-like methods Perry et al. [2025]. Karpov et al. study covert communication

and hidden reasoning in frontier models, while Zolkowski et al. find that current models show only early and limited steganographic capability without stronger coordination affordances Karpov et al. [2025], Zolkowski et al. [2025]. Our benchmark is closest in spirit to this safety-oriented line, but it tests a different capability: scheme-aware detection and extraction from diluted keyword signals over realistic cover corpora rather than coordinated message passing or hidden reasoning tasks.

Positioning our work. Across these threads, the missing benchmark is one that explicitly sweeps signal density and separates detection from recovery. That is the niche this paper fills. We ask whether current LLMs can still recover a message once the hiding rule remains simple in principle but becomes sparse, dispersed, and confounded by distractor triggers.

3 Methodology

Task definition. We study two tasks on the same passage. In **detection**, the model decides whether the passage hides a message under one of the candidate schemes and returns a binary judgment. In **extraction**, the model returns the six-letter secret or `NONE`. Separating the tasks matters because a monitor may detect suspicious structure without recovering the payload.

Benchmark construction. We build DILUTEDSTEGBENCH from three corpora with different context profiles: WIKITEXT-2 for long encyclopedia-style passages, IMDB for long reviews, and AG NEWS for shorter news passages. For each corpus, we synthesize three positive steganographic conditions and matched negatives:

- ACROSTIC: the first letter of each sentence encodes a six-letter secret.
- KEYWORD-DENSE: the first letter of the word immediately after the exact trigger token `signal` encodes the secret.
- KEYWORD-DILUTED: the same keyword-next-word rule is spread across a much longer passage and mixed with distractor variants such as `signals` and `signal-based`.

Each corpus-scheme cell contains four positive and four negative examples. This yields 72 synthetic examples in the main benchmark. We also run a small external sanity check on 24 examples from the `llm_steganalysis_benchmark` dataset, but we treat that probe as diagnostic rather than confirmatory because the prompt is generic and not aligned to the benchmark's native hiding schemes.

Models and prompts. We evaluate GPT-4.1 and GPT-5 through the OpenAI API OpenAI [2026a]. Prompting is fixed across runs. The detection prompt asks whether a hidden message is present under either candidate scheme. The extraction prompt asks for the six-letter message or `NONE`. Final prompt texts are stored in the project `prompts/` directory.

Metrics and statistics. For detection, we report accuracy, precision, recall, and F1. For extraction, we report exact six-letter recovery rate and character-level accuracy. We compute bootstrap 95% confidence intervals for per-cell metrics and use Fisher's exact test for pooled pairwise comparisons of exact recovery counts. Our primary hypothesis is directional: exact recovery should drop from ACROSTIC and KEYWORD-DENSE to KEYWORD-DILUTED, while detection should degrade more slowly.

Implementation details. Experiments use Python 3.12.8 with `openai`, `datasets`, `pandas`, `numpy`, `scipy`, `matplotlib`, `seaborn`, and `scikit-learn`. Four NVIDIA RTX A6000 GPUs were available but unused because all evaluation was API-based. Observed usage was 40,446 input and 3,968 output tokens for GPT-4.1, and 40,314 input and 5,460 output tokens for GPT-5. Using the official posted OpenAI pricing on April 19, 2026 OpenAI [2026b], the total estimated run cost was $0.2176.

Validation note. One implementation detail mattered for GPT-5. Early validation runs with `reasoning.effort="low"` exhausted the output budget with reasoning tokens and produced truncated JSON responses. We switched the final configuration to `reasoning.effort="minimal"` and updated the cache key before the final rerun. The cached validation rerun completed in roughly 14 seconds and reproduced the saved artifacts.

Model	Scheme	Det. Acc.	Precision	Recall	F1	Exact Rec.	Char. Acc.
GPT-4.1	ACROSTIC	0.875	0.909	0.833	0.870	0.667	0.861
GPT-4.1	KEYWORD-DENSE	**0.917**	0.857	1.000	**0.923**	**0.750**	**0.958**
GPT-4.1	KEYWORD-DILUTED	0.792	**1.000**	0.583	0.737	0.250	0.597
GPT-5	ACROSTIC	**0.958**	**1.000**	0.917	**0.957**	**0.583**	0.875
GPT-5	KEYWORD-DENSE	0.792	0.706	**1.000**	0.828	0.500	**0.889**
GPT-5	KEYWORD-DILUTED	0.750	0.667	**1.000**	0.800	0.167	0.486

Table 1: Pooled main results across corpora. Detection metrics are aggregated over 24 examples per model-scheme pair, with 12 positives and 12 negatives. Extraction metrics use 12 positive examples per model-scheme pair. Best results within each model are in **bold**. Dilution hurts exact recovery much more than detection.

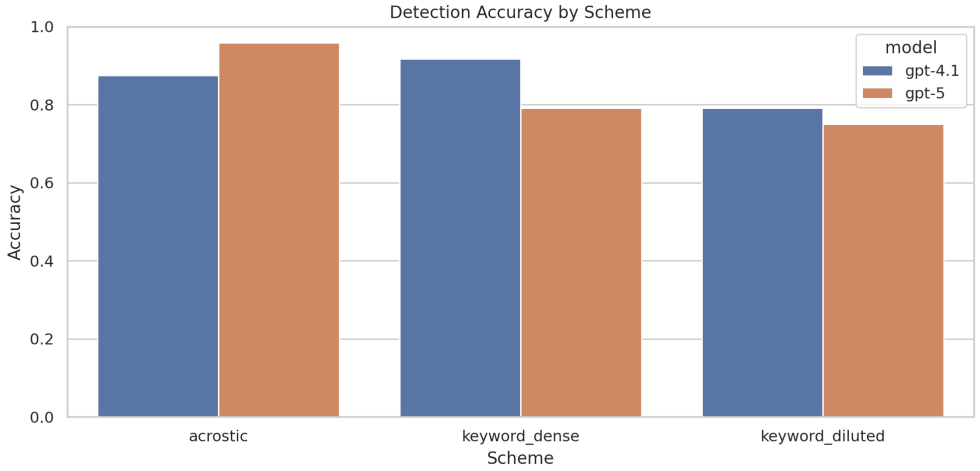

Figure 1: Detection accuracy by scheme and model. Detection remains relatively strong across all three schemes, although performance drops on diluted keyword signals. The gap between Table 1 and this figure is the main qualitative result of the paper: models can often notice suspicious structure before they can decode the secret exactly.

4 Results

Dilution hurts extraction more than detection. Table 1 shows the central pattern. For both models, KEYWORD-DILUTED is the hardest extraction condition by a wide margin. GPT-4.1 drops from 0.75 exact recovery on KEYWORD-DENSE to 0.25 on KEYWORD-DILUTED, while GPT-5 drops from 0.50 to 0.17. Detection also worsens under dilution, but the decline is smaller. This creates a clear detection-extraction gap, especially for GPT-4.1, which still reaches 0.79 detection accuracy on diluted passages.

Statistical comparisons. Table 2 reports Fisher exact tests on pooled exact-recovery counts. The strongest confirmatory result is the GPT-4.1 dense-versus-diluted comparison ($p = 0.039$), where exact recovery improves by 50 percentage points. The same directional trend appears in all other comparisons, but the small sample size leaves the remaining tests underpowered.

Corpus effects are real but not uniform. Table 3 shows that longer context does not automatically help. GPT-4.1 performs best on WIKITEXT-2 for ACROSTIC and KEYWORD-DENSE, but it still falls to 0.50 on diluted WIKITEXT-2. GPT-5 is even less stable: it reaches 0.75 exact recovery on WIKITEXT-2 acrostics yet drops to 0.00 on diluted WIKITEXT-2. This pattern suggests that longer passages help only when the hidden rule remains easy to keep in local working memory.

Confidence intervals and external sanity check. Per-cell bootstrap intervals are wide because each extraction cell contains only four positive examples and each detection cell contains eight total examples. Many extraction intervals therefore span most of the unit interval. The external sanity

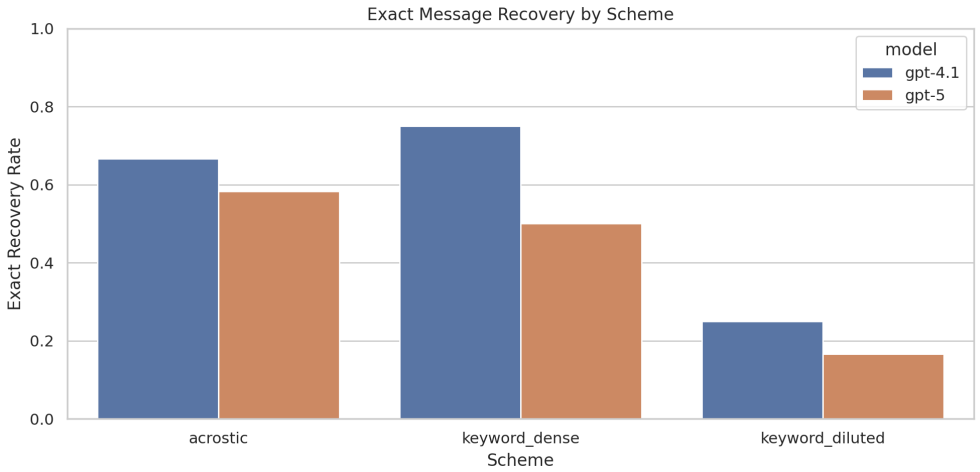

Figure 2: Exact message recovery by scheme and model. Both models follow the same ranking: KEYWORD-DENSE and ACROSTIC outperform KEYWORD-DILUTED. The drop is especially sharp for GPT-5, which reaches only 0.17 exact recovery in the diluted setting.

Model	Comparison	Rate A	Rate B	Delta	Fisher p
GPT-4.1	Acrostic vs. Diluted	0.667	0.250	0.417	0.100
GPT-4.1	Dense vs. Diluted	0.750	0.250	0.500	**0.039**
GPT-5	Acrostic vs. Diluted	0.583	0.167	0.417	0.089
GPT-5	Dense vs. Diluted	0.500	0.167	0.333	0.193

Table 2: Pairwise Fisher exact tests on pooled exact-recovery counts. Every comparison favors the less diluted scheme. Only the GPT-4.1 dense-versus-diluted comparison reaches the 0.05 threshold at this sample size.

check on 24 benchmark examples is near chance for both models and is not part of our main claim. We interpret it as a prompt-alignment limitation rather than evidence that the models cannot detect steganography in general.

5 Discussion

What do the results mean? The main result is not that dilution makes hidden structure invisible. It is that dilution breaks exact recovery before it breaks detection. That distinction matters for any monitor that relies on a prompted LLM alone. A model may correctly suspect that a passage is unusual, yet fail to reconstruct the payload needed for attribution, filtering, or automated response.

Why is GPT-4.1 stronger here? On this benchmark, GPT-4.1 is more reliable than GPT-5 for extraction, especially on KEYWORD-DENSE and KEYWORD-DILUTED. We do not treat this as a broad ranking claim between the models. The result is prompt- and API-specific, and it likely depends on how each model balances reasoning behavior with constrained JSON output. Still, under the exact protocol used here, GPT-4.1 preserves more payload information once the signal becomes sparse.

What failures dominate? The processed outputs show three recurring errors. First, diluted passages often trigger false negatives in extraction, with the model returning `NONE` despite detecting some suspicious structure. Second, near-miss decodes are common, including one-letter substitutions or transpositions such as `PRAIRA`, `QUARRY`, `EMBERB`, and `NELBUL`. Third, longer diluted passages become brittle when distractor forms compete with the true trigger. These are not random errors.

Model	Corpus	Acrostic	Dense	Diluted
GPT-4.1	AG NEWS	0.50	0.50	0.25
GPT-4.1	IMDB	0.50	0.75	0.00
GPT-4.1	WIKITEXT-2	**1.00**	**1.00**	**0.50**
GPT-5	AG NEWS	**0.75**	0.25	**0.25**
GPT-5	IMDB	0.25	**0.75**	**0.25**
GPT-5	WIKITEXT-2	**0.75**	0.50	0.00

Table 3: Exact recovery by corpus and scheme. Longer context does not uniformly help: WIKITEXT-2 is easiest for obvious schemes but still difficult in the diluted setting, especially for GPT-5. Best results within each model and scheme block are in **bold**.

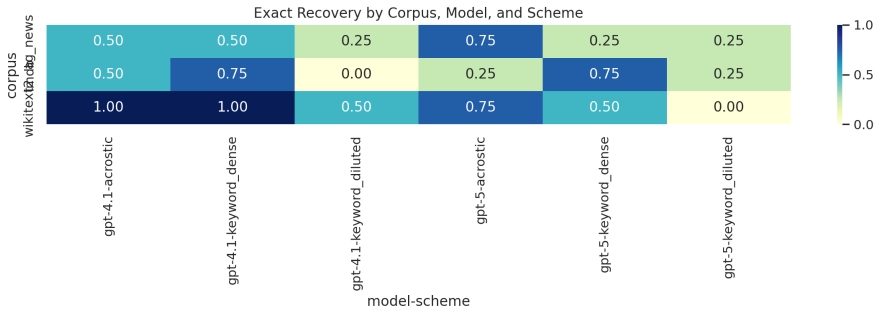

Figure 3: Exact recovery heatmap by corpus, scheme, and model. The heatmap highlights two consistent facts: diluted extraction is weak across corpora, and model performance interacts with corpus length in a non-monotonic way rather than improving uniformly with more context.

They are exactly the kinds of failures one would expect when a model tracks local clues but loses the global decoding path.

Limitations. This study is deliberately small and controlled. Each scheme-model comparison uses only 12 positive examples, so uncertainty remains high. The benchmark is synthetic, even though the cover text comes from real corpora, and our negatives are clean matched controls rather than a broad sample of natural text. The model is told the candidate hiding schemes, so this is not open-ended steganalysis. Finally, the GPT-5 validation fix shows that API behavior can materially affect results; our conclusions should therefore be read as claims about this benchmark, prompt set, and API configuration rather than universal model properties.

Broader implications. These findings sharpen the current safety picture from recent frontier-model studies Karpov et al. [2025], Zolkowski et al. [2025]. A sparse covert channel does not need to be fully undetectable to be operationally problematic. It may be enough that a monitor can flag suspicion but cannot reliably decode the message. That suggests future monitoring systems should combine prompted models with stronger search, rule induction, or programmatic decoding components rather than expecting a single LLM call to do both detection and extraction robustly.

6 Conclusion

We presented DILUTEDSTEGBENCH, a corpus-grounded benchmark for diluted textual steganography that separates binary detection from exact message recovery. Across WIKITEXT-2, IMDB, and AG NEWS, both GPT-4.1 and GPT-5 handled obvious and dense schemes substantially better

than diluted keyword-triggered encoding. The central finding is simple: dilution reduces extraction reliability sharply before detection collapses.

The main takeaway is that current LLM-based monitors can often sense that a passage is suspicious without recovering the payload that explains why. Future work should increase sample size, test more model families, vary message length and trigger ambiguity independently, and compare scheme-aware prompts with open-ended steganalysis. A stronger next benchmark should also add paraphrase and edit attacks to measure whether diluted signals remain fragile once the cover text is perturbed.

References

Ziqi Bai, Yijia Yang, Zhen Pang, Jialin Wang, and Yongfeng Huang. Towards next-generation steganalysis: Llms unleash the power of detecting steganography. *arXiv preprint arXiv:2405.09090*, 2024. URL https://arxiv.org/abs/2405.09090.

Anton Karpov, David Adeleke, Hyunwoo Cho, and Marta Perez-Campanero. The steganographic potentials of language models. *arXiv preprint arXiv:2505.03439*, 2025. URL https://arxiv.org/abs/2505.03439.

Zichen Lin, Leonardo Luo, Han Zhang, and Jun Luo. Zero-shot generative linguistic steganography. *arXiv preprint arXiv:2403.10856*, 2024. URL https://arxiv.org/abs/2403.10856.

OpenAI. Models: GPT-5 and GPT-4.1. https://developers.openai.com/api/docs/models/gpt-5, 2026a. Accessed: 2026-04-19.

OpenAI. Api pricing. https://openai.com/api/pricing/, 2026b. Accessed: 2026-04-19.

Nelson Perry, Karan Gupte, Davide Pitta, and Dror Rotem. Robust steganography from large language models. *arXiv preprint arXiv:2504.08977*, 2025. URL https://arxiv.org/abs/2504.08977.

Koushik Raj-Sankar and Aditya Rajagopalan. Hiding in plain sight: Towards the science of linguistic steganography. *arXiv preprint arXiv:2312.16840*, 2023. URL https://arxiv.org/abs/2312.16840.

Rintaro Ueoka, Yugo Murawaki, and Sadao Kurohashi. Frustratingly easy edit-based linguistic steganography with a masked language model. *arXiv preprint arXiv:2104.09833*, 2021. URL https://arxiv.org/abs/2104.09833.

Jinlong Wu, Si Wu, Chenyang Xue, Qi Wen, and Li Peng. Generative text steganography with large language model. *arXiv preprint arXiv:2404.10229*, 2024. URL https://arxiv.org/abs/2404.10229.

Shang Yi, Yanzhou Wu, Zhiyuan Feng, and Xinpeng Zhang. Exploiting language model for efficient linguistic steganalysis. *arXiv preprint arXiv:2107.12168*, 2021. URL https://arxiv.org/abs/2107.12168.

Ru Zhang, Jinchao Yang, Fei Yang, and Yongfeng Huang. Provably secure generative linguistic steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3046–3055, 2021. URL https://aclanthology.org/2021.findings-acl.268/.

Nils Zolkowski, Simon Nishimura-Gasparian, Niall McCarthy, Rasmus Zimmermann, and David Lindner. Early signs of steganographic capabilities in frontier llms. *arXiv preprint arXiv:2507.02737*, 2025. URL https://arxiv.org/abs/2507.02737.